

Topological Sort Using DFS

Pseudocode:

TOPOLOGICAL SORT(G)

mark all vertices as UNVISITED

stack = empty

for each vertex v

if v is UNVISITED

if DFS(v) = CYCLE

print "Cycle detected"

return

print stack in reverse order

DFS(v)

mark v as VISITING

for each neighbour u

if u is VISITING

return CYCLE

if u is UNVISITED

if DFS(u) = CYCLE

return CYCLE

mark v as VISITED

push v into stack

return NO_CYCLE

Idea:

if DFS reaches a VISITING vertex, a cycle exists
Otherwise, reverse the stack to get the topological order

2) MST with Constraints

CONSTRAINED - MST (A, Mandatory, Forbidden, Budget)

MST = empty

Cost = 0

Add all Mandatory edges

If they form a cycle $\rightarrow$ return "No MST"

Sort remaining edges by weight

For each edge e

if e is Forbidden

skip

if adding e creates a cycle

skip

add e to MST

cost += e . weight

if Cost > Budget

return "Budget exceeded"

if MST has $V-1$ edges

return MST

return "No feasible MST"

Idea:

Add mandatory edges first, ignore forbidden edges, then add the cheapest valid edges while staying within budget.